LAPORAN TUCIL 3 STIMA

A, Uniform Cost Search, Greedy Best First Search*

Strategi Algoritma - IF2211

Faiq Azzam Nafidz (13524003)

Billie Bhaskara Wibawa (13524024)

10.05.2026

DAFTAR ISI

DAFTAR ISI **1**
GARIS BESAR PROYEK **2**
PENJELASAN ALGORITMA **2**
Algoritma Uniform Cost Search 2
Algoritma Greedy Best First Search 4
Algoritma A* 7
Algoritma BFS 9
Algoritma Beam Search 11
Perbandingan Algoritma 16
PENGUJIAN **18**
Test Case 1 18
Test Case 2 27
Test Case 3 35
Test Case 4 45
Test Case 5 51
ANALISIS PENGUJIAN **68**
KESIMPULAN **69**
SARAN **69**
PERNYATAAN INTEGRITAS **70**
LAMPIRAN **71**
TANGKAPAN LAYAR **72**

GARIS BESAR PROYEK

Pengerjaan kami lakukan sepenuhnya menggunakan C++. Kami menggunakan library GUI dengan nama Dear ImGui. Library ini cukup lightweight dan penulis secara pribadi juga ingin mempelajarinya demi *project* lain, jadi sekalian digunakan di tugas kali ini. Kali ini tidak ada arsitektur rumit dalam bentuk apapun. It's just a straightforward algorithm and GUI software.

PENJELASAN ALGORITMA

Tugas kecil 3 dalam mata kuliah Strategi Algoritma kali ini membahas gim bernama Ice Sliding Puzzle, yaitu permainan logika yang di dalamnya pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Tugas ini mengharuskan untuk mencari solusi dari permainan tersebut. Maka dari itu, dibuatlah program untuk melakukan pencarian solusi menggunakan berbagai algoritma penentuan rute (*pathfinding*). Algoritma yang telah diimplementasikan adalah algoritma *uniform cost search* (*UCS*), *greedy best first search* (*GBFS*), dan *A**.

Algoritma *Uniform Cost Search*

Uniform Cost Search merupakan salah satu algoritma penentuan rute yang bertipe *uninformed search* (tidak diberikan informasi tambahan di luar permasalahan yang ada). Dalam suatu graf, algoritma ini menelusuri simpul dari daftar simpul yang ingin ditelusuri yang memiliki nilai biaya (*cost*) terkecil terlebih dahulu dengan meng-*expand* (memperluas/mengembangkan) simpul tersebut ke simpul tetangga-tetangganya untuk dimasukkan pada daftar simpul tersebut.

Perhitungan nilai biaya yang pada tiap simpul dihitung dengan fungsi g(n), yaitu fungsi yang mengembalikan biaya akumulatif dari awal simpul hingga simpul yang sedang ditelusuri. Dengan begitu, algoritma dapat mengetahui simpul terbaik mana yang perlu ditelusuri dan dikembangkan (yaitu simpul dengan biaya terkecil).

Algoritma ini selalu menghasilkan solusi yang optimal pada kasus gim ini karena tidak ada biaya negatif yang mungkin ada di suatu pergerakan dalam gim. Dengan itu, setiap pergerakan yang dipilih akan selalu menambah akumulasi biaya pada suatu simpul hasil pengembangan. Oleh karena itu, pilihan simpul dengan biaya terkecil adalah pilihan terbaik karena kemungkinan biaya total ketika menelusuri simpul tersebut adalah biaya simpul itu sendiri atau lebih. Pemilihan simpul dengan biaya lebih tinggi akan

menyebabkan kemungkinan terbaik yang bisa didapatkan tidak ditelusuri lebih lanjut.

Cuplikan kode:

```
std::pair<std::pair<int, std::vector<Direction>>, std::pair<int,
std::vector<std::pair<int, std::vector<Direction>>>>> UCS(const Board& board,
bool isOrdered){
    SearchNode start(board.pinX, board.pinY, board.ord);
    SearchNode goal(board.winX, board.winY, board.highestOrd+1);
    if(!isOrdered) goal.ord = -1;
    std::priority_queue<Neighbor, std::vector<Neighbor>,
std::greater<Neighbor>> pq;
    std::unordered_map<SearchNode, double, SearchNodeHash> dist;
    std::unordered_map<SearchNode, std::pair<SearchNode, Direction>,
SearchNodeHash> parent;


    pq.push({start, 0.0, 0.0, Direction::RIGHT});

    dist[start] = 0.0;

    int countIterate = 0;
    std::vector<std::pair<int, std::vector<Direction>>> iteration = {};

    while (!pq.empty()) {

        Neighbor current = pq.top();
        pq.pop();
        countIterate++;

        SearchNode currentNode = current.node;
        int currentCost = current.totalCost;
        iteration.push_back({currentCost, reconstructPath(parent, start,
currentNode)});

        if (currentNode == goal) {
            return {{currentCost, reconstructPath(parent, start, currentNode)},
{countIterate, iteration}};
```

```
        }

        auto expanded = expand(board, currentNode, currentCost);

        for (const Neighbor& next : expanded) {
            if (!dist.count(next.node) || next.totalCost < dist[next.node]) {
                dist[next.node] = next.totalCost;
                parent[next.node] = {currentNode, next.dir};
                pq.push({next.node, next.totalCost, next.totalCost, next.dir});
            }
        }
    }

    return {{-1, {}}, {-1, {}}};
}
```

Algoritma *Greedy Best First Search*

Greedy best first search (*BCFS*) merupakan algoritma penentuan rute bertipe *informed search* (ada informasi tambahan di luar permasalahan), berbeda dengan *UCS*. Algoritma ini memerlukan penetapan heuristik (yang merupakan informasi tambahan di luar permasalahan) untuk memprediksi simpul mana yang terbaik untuk ditelusuri. Heuristik yang ditetapkan memberikan nilai h(n) sebagai biaya estimasi suatu simpul ke simpul tujuan.

Proses algoritma ini mirip sekali dengan *UCS*; kembangkan simpul dengan biaya terkecil ke tetangga-tetangganya. Namun, biaya tersebut diganti yang awalnya adalah g(n) diganti menjadi h(n). Hal ini dilakukan terus-menerus sampai ditelusuri simpul yang merupakan tujuan. Dengan begitu, tiap langkah dari *GBFS* akan selalu mendekati simpul tujuan karena selalu memilih biaya terkecil yang diestimasi.

Heuristik yang digunakan bisa bermacam-macam sesuai dengan konteks permasalahan. Dalam konteks gim Ice Sliding Puzzle ini, diimplementasikan heuristik sebagai berikut.

1. Manhattan Distance

   Heuristik ini mengatakan bahwa petak terdekat dengan petak tujuan berdasarkan *manhattan distance* adalah petak yang terbaik untuk ditelusuri. *Manhattan distance* sendiri adalah jarak yang dihitung dengan menjumlahkan

selisih nilai absis dan selisih nilai ordinat antara dua posisi pada suatu bidang.

$$manhattan\ distance\ =\ \Delta x\ +\ \Delta y$$

2. Euclidean Distance

   Mirip seperti heuristik sebelumnya, heuristik ini mengatakan bahwa petak terdekat dengan petak tujuan adalah petak terbaik untuk ditelusuri, namun berdasarkan *euclidean distance*. *Euclidean distance* sendiri adalah jarak yang dihitung dengan menjumlahkan kuadrat dari selisih nilai absis dan selisih nilai ordinat lalu mengakarkan hasilnya.

$$euclidean\ distance\ =\ \sqrt{(\Delta x)^2\ +\ (\Delta y)^2}$$

3. Chebyshev Distance

   Sangat mirip dengan heuristik Manhattan Distance, namun jarak dihitung berdasarkan *chebyshev distance*, yaitu jarak yang dihitung dengan mencari selisih terbesar antara selisih nilai absis atau selisih nilai ordinat.

$$chebyshev\ distance\ =\ max(\Delta x,\ \Delta y)$$

Algoritma ini, dengan ketiga heuristik di atas, tidak selalu menghasilkan solusi optimal. Hal ini karena bisa saja jalur yang langsung menuju ke tujuan memiliki biaya yang lebih besar dibanding jalur alternatifnya. Misal, pergerakan ke atas akan langsung menuju ke tujuan, namun terdapat petak dengan biaya 100 di jalur tersebut, sedangkan pergerakan ke kanan, atas, lalu kiri memiliki total biaya hanya 3, jauh di bawah solusi sebelumnya. Bahkan, algoritma ini terkadang juga tidak bisa mencari solusi pada konfigurasi gim bersolusi. Algoritma ini sangat rentan terjebak pada simpul *dead-end* (simpul yang tidak bisa dikembangkan lebih jauh). Sebagai contoh, algoritma dapat menentukan bahwa pergerakan ke atas adalah pergerakan terbaik. Namun, setelah pergerakan tersebut, tidak ada gerakan lain yang bisa dilakukan kecuali kembali ke posisi semula. Saat kembali ke posisi semula, algoritma menentukan kembali bahwa pergerakan ke atas adalah pergerakan terbaik. Hal ini terus menerus terjadi sehingga algoritma tidak bisa mendapatkan solusi.

Cuplikan kode:

```
std::pair<std::pair<int, std::vector<Direction>>, std::pair<int,
std::vector<std::pair<int, std::vector<Direction>>>>> GBFS(const Board& board,
int heuristic, bool isOrdered){
```

```
    SearchNode start(board.pinX, board.pinY, board.ord);
    SearchNode goal(board.winX, board.winY, board.highestOrd+1);
    if(!isOrdered) goal.ord = -1;
    std::priority_queue<Neighbor, std::vector<Neighbor>,
std::greater<Neighbor>> pq;
    std::unordered_map<SearchNode, double, SearchNodeHash> dist;
    std::unordered_map<SearchNode, std::pair<SearchNode, Direction>,
SearchNodeHash> parent;

    // GBFS: Priority is h(n)
    pq.push({start, 0.0, heuristics(start, goal, heuristic),
Direction::RIGHT});

    dist[start] = 0.0;

    int countIterate = 0;
    std::vector<std::pair<int, std::vector<Direction>>> iteration = {};

    while (!pq.empty()) {

        Neighbor current = pq.top();
        pq.pop();
        countIterate++;

        SearchNode currentNode = current.node;
        int currentCost = current.totalCost;
        iteration.push_back({currentCost, reconstructPath(parent, start,
currentNode)});

        if (currentNode == goal) {
            return {{currentCost, reconstructPath(parent, start, currentNode)},
{countIterate, iteration}};
        }

        auto expanded = expand(board, currentNode, currentCost);
```

```
        for (const Neighbor& next : expanded) {
            double estimatedCostOnward = heuristics(next.node, goal,
heuristic);
            if (!dist.count(next.node) || estimatedCostOnward <
dist[next.node]) {
                dist[next.node] = estimatedCostOnward;
                parent[next.node] = {currentNode, next.dir};
                pq.push({next.node, next.totalCost, estimatedCostOnward,
next.dir});
            }
        }
    }

    return {{-1, {}}, {-1, {}}};
}
```

Algoritma *A**

*A** secara sederhana menggabungkan kedua algoritma *UCS* dan *GBFS*. Prosesnya sangat mirip dengan kedua algoritma sebelumnya, namun perhitungan biaya dihitung berdasarkan akumulasi biaya dari awal simpul (fungsi g(n)) ditambah estimasi biaya berdasarkan heuristik yang digunakan (fungsi h(n)). Dengan begitu, perhitungan biaya pada algoritma ini adalah menggunakan fungsi f(n) = g(n) + h(n). Algoritma ini menutupi kelemahan *UCS* yang tidak memiliki arah pengembangan yang jelas dan menutupi kelemahan *GBFS* yang tidak selalu memberikan solusi optimal.

Pada konteks gim, *A** menggunakan heuristik yang sama seperti yang ada pada algoritma *GBFS*. Algoritma ini, dengan heuristik yang digunakan, bersifat *admissible* (selalu menghasilkan solusi optimal) jika biaya tiap petak yang dilewati lebih dari atau sama dengan satu. Hal ini karena nilai yang didapat dari heuristik ini selalu kurang dari atau sama dengan nilai sebenarnya (untuk heuristik Manhattan Distance, kesamaan didapat ketika semua petak pada jalur memiliki biaya 1 semua). Namun, karena petak pada gim ini bisa memiliki nilai 0, algoritma ini menjadi tidak *admissible*. Solusi terbaik tidak didapatkan menggunakan heuristik ini ketika pergerakan *direct* ke tujuan (pergerakan yang mengurangi jarak ke tujuan secara langsung) memiliki petak berbiaya lebih dari 0, namun terdapat jalur *indirect* (tidak melakukan pergerakan yang secara

langsung mengurangi jarak antara posisi awal ke tujuan) yang semua petaknya berbiaya 0.

Cuplikan kode:

```
std::pair<std::pair<int, std::vector<Direction>>, std::pair<int,
std::vector<std::pair<int, std::vector<Direction>>>>> ASTAR(const Board& board,
int heuristic, bool isOrdered){
    SearchNode start(board.pinX, board.pinY, board.ord);
    SearchNode goal(board.winX, board.winY, board.highestOrd+1);
    if(!isOrdered) goal.ord = -1;
    std::priority_queue<Neighbor, std::vector<Neighbor>,
std::greater<Neighbor>> pq;
    std::unordered_map<SearchNode, double, SearchNodeHash> dist;
    std::unordered_map<SearchNode, std::pair<SearchNode, Direction>,
SearchNodeHash> parent;

    // A*: Priority is f(n) = g(n) + h(n)
    pq.push({start, 0.0, heuristics(start, goal, heuristic),
Direction::RIGHT});

    dist[start] = 0.0;

    int countIterate = 0;
    std::vector<std::pair<int, std::vector<Direction>>> iteration = {};

    while (!pq.empty()) {

        Neighbor current = pq.top();
        pq.pop();
        countIterate++;

        SearchNode currentNode = current.node;
        int currentCost = current.totalCost;
        iteration.push_back({currentCost, reconstructPath(parent, start,
currentNode)});

        if (currentNode == goal) {
```

```
            return {{currentCost, reconstructPath(parent, start, currentNode)},
{countIterate, iteration}};
        }

        auto expanded = expand(board, currentNode, currentCost);

        for (const Neighbor& next : expanded) {
            double estimatedCostOnward = heuristics(next.node, goal,
heuristic);
            if (!dist.count(next.node) || next.totalCost + estimatedCostOnward
< dist[next.node]) {
                dist[next.node] = next.totalCost + estimatedCostOnward;
                parent[next.node] = {currentNode, next.dir};
                pq.push({next.node, next.totalCost, next.totalCost +
estimatedCostOnward, next.dir});
            }
        }
    }

    return {{-1, {}}, {-1, {}}};
}
```

Algoritma *BFS*

Breadth First Search adalah algoritma yang mengecek semua kemungkinan path sampai menemukan *goal*-nya. Beda dengan implementasi algoritma lainnya pada tucil ini, BFS sama sekali tidak menghitung *cost* dari jalannya. Melainkan algoritma ini meminimalisir jumlah *slide* yang dilakukan oleh pin. Terlihat pada kutipan kode:

```
            if (visited.find(next.node) == visited.end()) {

                visited.insert(next.node);

                parent[next.node] = {currentNode, next.dir};

                q.push(next);

            }
```

BFS tidak menggunakan *priority queue* seperti algoritma lainnya. Melainkan di sini *BFS* hanya menggunakan *queue* biasa. Dalam *BFS*, tiap *node* dalam *queue* akan di-*expand* sesuai dengan urutannya dimasukkannya dalam *queue.* Proses ini dilakukan sampai ditemukan *goal node*. Yang apabila ditemukan, maka dilakukan konstruksi *path* menggunakan fungsi *helper* yang sama seperti yang digunakan di algoritma lainnya.

Untuk BFS maka tentunya g(n) dan h(n) tidak ada sama sekali dalam proses pemilihan *node* untuk di-*expand.* Meski begitu, g(n) (*actual cost*) tetap saja dihitung dalam program ini hanya untuk mencari tahu *cost* dari solusi yang didapatkannya.

Cuplikan kode:

```
std::pair<std::pair<int, std::vector<Direction>>, std::pair<int,
std::vector<std::pair<int, std::vector<Direction>>>>> BFS(const Board& board,
bool isOrdered){
    /*
    This is just.. BFS I guess. But an interesting result is that it
    optimizes the number of moves, not the total cost
    */
    SearchNode start(board.pinX, board.pinY, board.ord);
    SearchNode goal(board.winX, board.winY, board.highestOrd+1);
    if(!isOrdered) goal.ord = -1;
    std::queue<Neighbor> q;
    std::unordered_set<SearchNode, SearchNodeHash> visited;
    std::unordered_map<SearchNode, std::pair<SearchNode, Direction>,
SearchNodeHash> parent;
    q.push({start, 0.0, 0.0, Direction::RIGHT});
    visited.insert(start);

    int countIterate = 0;
    std::vector<std::pair<int, std::vector<Direction>>> iteration = {};

    while (!q.empty()) {

        Neighbor current = q.front();
        q.pop();
        countIterate++;
```

```
        SearchNode currentNode = current.node;
        int currentCost = current.totalCost;
        iteration.push_back({currentCost, reconstructPath(parent, start,
currentNode)});
        if (currentNode == goal) {
            return {{currentCost, reconstructPath(parent, start, currentNode)},
{countIterate, iteration}};
        }

        auto expanded = expand(board, currentNode, currentCost);
        for (const Neighbor& next : expanded) {

            if (visited.find(next.node) == visited.end()) {
                visited.insert(next.node);
                parent[next.node] = {currentNode, next.dir};
                q.push(next);
            }
        }
    }

    return {{-1, {}}, {-1, {}}};
}
```

Algoritma *Beam Search*

Beam Search yang kami implementasikan di program ini adalah variasi dari A* (atau UCS apabila tidak memilih *heuristic* apapun). Modifikasi yang dilakukan adalah dengan membatasi jumlah *node* yang dievaluasi agar selalu di bawah nilai k yang didefinisikan. Contohnya dalam algoritma A* standar jumlah *node* yang di-*expand* tidak dibatasi sama sekali. Dalam papan yang besar ini bisa sangat banyak mengkonsumsi *memory* dan menghasilkan waktu proses yang lama.

Dalam algoritma *Beam Search*, *priority queue* dari *nodes*-nya akan selalu dilakukan pengecekan tiap iterasi, dan apabila panjangnya melebihi *beam* yang selebar k, semua *node* di luar beam akan dibuang secara permanen. Ini membuat algoritma *Beam Search* tidak pernah mengkonsumsi *memory* lebih dari batasan k yang diberikan. Hal ini juga

membuat proses pencarian rute menjadi lebih cepat di beberapa kasus *board* yang sangat besar.

Penjelasan di bawah menunjukkan proses pemotongan *priority queue.*

```
while (!currentLevel.empty()) {

...

}
```

Terlihat bahwa *loop* utama menggunakan *priority queue* yang bernama "currentLevel". Namun pada kutipan berikutnya:

```
if (!dist.count(next.node) || f_n < dist[next.node]) {

        dist[next.node] = f_n;

        parent[next.node] = {currentNode, next.dir};

        nextLevel.push_back({next.node, next.totalCost, f_n, next.dir});

}
```

Namun di sini terlihat bahwa *neighbor nodes* dari *current node* yang telah di-*expand* justru dimasukkan ke *priority queue* lain bernama "nextLevel". Hal ini merupakan optimisasi karena *beam search* melakukan banyak penghapusan, maka menghapus *nodes* dalam satu *priority queue* berkali-kali akan membuat butuh dilakukannya banyak *shifting* elemen-elemen lainnya yang tidak terhapus. Sedangkan membuat 2 buah *priority queue*, kita bisa hanya push *nodes* secara runtut ke *priority queue* yang baru, dan membuang *priority queue* yang lama setelahnya. Hal ini terlihat dalam kutipan berikut:

```
if (nextLevel.size() > k) {

            std::nth_element(

                nextLevel.begin(),

                nextLevel.begin() + k,

                nextLevel.end(),

                [](const Neighbor& a, const Neighbor& b) {
```

```
                    return a.priority < b.priority;
                }
            );
            nextLevel.resize(k);
}
currentLevel = std::move(nextLevel);
```

Di sini kami menggunakan *method* dari *standard library C++* bernama "nth_element". *Method* ini gunanya adalah mencari element dengan urutan ke-n dari suatu *container* (dalam kasus ini *container* kita adalah *priority queue* yang menyimpan *expanded neighbors*). Namun efek samping bagus dari *method* ini adalah ia menggunakan proses yang sama dengan partisi *quick sort,* yang mana dipilih sebuah *pivot* dan semua elemen yang lebih kecil dari pivot itu di taruh di kirinya. Karena proses ini dilakukan, maka otomatis apabila kita menggunakan *method* ini untuk mencari elemen ke-n, maka otomatis semua elemen di kiri elemen ke-n pasti lebih kecil dari elemen ke-n karena efek samping dari proses *partition* tadi.

Note: *method nth_element* dapat diisi *lambda* untuk mendefinisikan proses *partitioning*-nya. Pada kutipan di atas diberi fungsi *lamba* yang membuat semua *node* dengan *priority* lebih tinggi dari *pivot* ada di kirinya.

Terakhir, ada kutipan kode:

```
currentLevel = std::move(nextLevel);
```

Kode ini merupakan optimisasi. Karena bisa saja kita menggunakan:

```
currentLevel = nextLevel;
```

Namun artinya ini akan membuat C++ meng-*copy* semua elemen dari "nextLevel" ke "currentLevel". Oleh karena itu dengan menggunakan *std::move,* maka *pointer* dari nextLevel dipindah ke currentLevel. Dengan menggunakan *std::move* ini juga maka secara *default* akan dipanggil *destructor* ke "currentLevel" yang lama. Dengan ini tidak lagi dilakukan *copy* dengan O(N). Melainkan prosesnya berjalan dengan waktu konstan.

Perlu dipahami bahwa dibandingkan A*, algoritma ini *on average* lebih cepat dalam

menemukan solusi, tapi tidak ada jaminan bahwa solusi yang ditemukan adalah solusi dengan *cost* paling rendah. Juga perlu diketahui apabila solusi merupakan bagian dari *search node* yang dibuang dalam proses pemotongan *priority queue*, maka algoritma ini dapat gagal menemukan solusi di papan yang mana harusnya mungkin untuk ditemukan sebuah solusi.

Cuplikan kode:

```
std::pair<std::pair<int, std::vector<Direction>>, std::pair<int,
std::vector<std::pair<int, std::vector<Direction>>>>> BeamSearch(const Board&
board, int heuristic, bool isOrdered, size_t k = 50){
    SearchNode start(board.pinX, board.pinY, board.ord);
    SearchNode goal(board.winX, board.winY, board.highestOrd+1);
    if(!isOrdered) goal.ord = -1;

    /*
    Cuz this algorithm is basically A* but limits the nodes considered into a
specified
    length, it uses a lot of discarding memories from vectors. Having one
vector and needing
    to shift all the object to the right from one pop is just too slow. So
instead we just
    push all the valid objects (in this case, the top k nodes) into a new
vector "nextLevel"
    and just discard the previous vector (effectively throwing it to the
std::vector dtor)
    */
    std::vector<Neighbor> currentLevel;
    std::unordered_map<SearchNode, double, SearchNodeHash> dist;
    std::unordered_map<SearchNode, std::pair<SearchNode, Direction>,
SearchNodeHash> parent;

    double startHeuristic = heuristics(start, goal, heuristic);
    currentLevel.push_back({start, 0.0, startHeuristic, Direction::RIGHT});
    dist[start] = 0.0;

    int countIterate = 0;
    std::vector<std::pair<int, std::vector<Direction>>> iteration = {};
```

```
    while (!currentLevel.empty()) {
        std::vector<Neighbor> nextLevel;
        for (const Neighbor& current : currentLevel) {
            countIterate++;
            SearchNode currentNode = current.node;
            int currentCost = current.totalCost;
            iteration.push_back({currentCost, reconstructPath(parent, start,
currentNode)});
            if (currentNode == goal) {
                return {{currentCost, reconstructPath(parent, start,
currentNode)}, {countIterate, iteration}};
            }

            auto expanded = expand(board, currentNode, currentCost);

            for (const Neighbor& next : expanded) {
                double estimatedCostOnward = heuristics(next.node, goal,
heuristic);
                double f_n = next.totalCost + estimatedCostOnward;

                if (!dist.count(next.node) || f_n < dist[next.node]) {

                    dist[next.node] = f_n;
                    parent[next.node] = {currentNode, next.dir};

                    nextLevel.push_back({next.node, next.totalCost, f_n,
next.dir});
                }
            }
        }

        /*
        This function is like quicksort where it picks a pivot and sorts
        everything less than it to the left and more than it to the right
        (the partitioning process). But it doesn't sort all the way, it just
```

```
        finds the nth element. But in the process of finding the nth element
        it also made it that every element to the left of the nth element
        become the top n elements. So we use this here.
        */
        if (nextLevel.size() > k) {
            std::nth_element(
                nextLevel.begin(),
                nextLevel.begin() + k,
                nextLevel.end(),
                [](const Neighbor& a, const Neighbor& b) {
                    return a.priority < b.priority;
                }
            );
            nextLevel.resize(k);
        }
        currentLevel = std::move(nextLevel);
    }
    return {{-1, {}}, {-1, {}}};
}
```

Perbandingan Algoritma

Terdapat beberapa faktor penentu kebagusan algoritma penentu rute. Di bagian ini, akan dibandingkan setiap algoritma yang sudah dijelaskan berdasarkan kekomplitan, keoptimalan solusi, dan keefisienan algoritma. Algoritma *UCS* merupakan algoritma yang komplit dan optimal karena akan meninjau seluruh simpul yang mungkin dan selalu memilih simpul terbaik pada iterasi tertentu. Algoritma *A** juga memiliki atribut yang sama seperti *UCS*, dengan pengecualian ketika terdapat petak yang bernilai 0 (lihat penjelasan sebelumnya) sehingga terdapat kasus algoritma *A** menghasilkan solusi nonoptimal. Algoritma *GBFS* dan *Beam Search* mengedepankan efisiensi pencarian dibandingkan kekomplitan dan keoptimalan solusi, dengan *Beam Search* lebih efisien ketika nilai *k* (batasan lebar pencarian dalam satu level) semakin kecil (namun juga mengorbankan lebih lanjut kekomplitan solusi). Terakhir, *algoritma BFS* bersifat komplit (menelusuri seluruh simpul yang mungkin) namun tidak selalu optimal (karena tidak menghiraukan biaya petak, hanya berapa petak yang dikunjungi yang dihiraukan). Jika di-*ranking* kelima algoritma tersebut dari 1–5 (dengan 5 adalah yang terbaik dan 1 adalah yang terburuk), akan seperti tabel berikut.

	Komplit	Optimal	Efisien
UCS	5	5	2
GBFS	3	1	4
*A**	5	4	3
BFS	5	3	1
Beam Search	4	2	5

Sederhananya, Jika ingin pencarian komplit dan solusi optimal tanpa ada petak berbiaya 0, pilihlah *A**. Jika ingin pencarian komplit dan optimal namun terdapat petak berbiaya 0, pilihlah *UCS*. Jika ingin pencarian efisien, pilihlah *Beam Search* dengan nilai k paling kecil yang membuahkan solusi (sesuaikan dengan kondisi papan gim).

Test Case 1

Input:

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999
```

Output:

UCS	`Solusi Yang Ditemukan : RULUDRUR`
	`Cost dari solusi : 87`
	
	`Initial`
	`XXXXXXX`
	`X0....X`
	`X..X..X`
	`X....OX`
	`X1...LX`
	`XZ..X.X`
	`XXXXXXX`
	
	`STEP 1 : R`
	`XXXXXXX`
	`X0....X`
	`X..X..X`
	`X....OX`
	`X1...LX`
	`X..ZX.X`
	`XXXXXXX`
	
	`STEP 2 : U`

```
XXXXXXX
X0....X
X..X..X
X..Z.OX
X1...LX
X...X.X
XXXXXXX

STEP 3 : L
XXXXXXX
X0....X
X..X..X
XZ...OX
X1...LX
X...X.X
XXXXXXX

STEP 4 : U
XXXXXXX
XZ....X
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 5 : D
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
XZ..X.X
XXXXXXX

STEP 6 : R
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
X..ZX.X
XXXXXXX

STEP 7 : U
XXXXXXX
X0....X
```

	X..X..X X..Z.OX X1...LX X...X.X XXXXXXX STEP 8 : R XXXXXXX X0....X X..X..X X....ZX X1...LX X...X.X XXXXXXX Waktu eksekusi: 0.0559 ms Banyak iterasi yang dilakukan: 13 iterasi
GBFS	Solusi Yang Ditemukan : RURULDRUR Cost dari solusi : 104 Initial XXXXXXX X0....X X..X..X X....OX X1...LX XZ..X.X XXXXXXX STEP 1 : R XXXXXXX X0....X X..X..X X....OX X1...LX X..ZX.X XXXXXXX STEP 2 : U XXXXXXX X0....X X..X..X X..Z.OX X1...LX X...X.X XXXXXXX

```
STEP 3 : R
XXXXXXX
X0....X
X..X..X
X....ZX
X1...LX
X...X.X
XXXXXXX

STEP 4 : U
XXXXXXX
X0...ZX
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 5 : L
XXXXXXX
XZ....X
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 6 : D
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
XZ..X.X
XXXXXXX

STEP 7 : R
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
X..ZX.X
XXXXXXX

STEP 8 : U
```

	XXXXXXX X0....X X..X..X X..Z.OX X1...LX X...X.X XXXXXXX STEP 9 : R XXXXXXX X0....X X..X..X X....ZX X1...LX X...X.X XXXXXXX Waktu eksekusi: 0.108911 ms Banyak iterasi yang dilakukan: 12 iterasi
A*	Solusi Yang Ditemukan : RULUDRUR Cost dari solusi : 87 Initial XXXXXXX X0....X X..X..X X....OX X1...LX XZ..X.X XXXXXXX STEP 1 : R XXXXXXX X0....X X..X..X X....OX X1...LX X..ZX.X XXXXXXX STEP 2 : U XXXXXXX X0....X X..X..X X..Z.OX X1...LX

```
X...X.X
XXXXXXX

STEP 3 : L
XXXXXXX
X0....X
X..X..X
XZ...OX
X1...LX
X...X.X
XXXXXXX

STEP 4 : U
XXXXXXX
XZ....X
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 5 : D
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
XZ..X.X
XXXXXXX

STEP 6 : R
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
X..ZX.X
XXXXXXX

STEP 7 : U
XXXXXXX
X0....X
X..X..X
X..Z.OX
X1...LX
X...X.X
XXXXXXX
```

	STEP 8 : R XXXXXXX X0....X X..X..X X....ZX X1...LX X...X.X XXXXXXX Waktu eksekusi: 0.480391 ms Banyak iterasi yang dilakukan: 13 iterasi
BFS	Solusi Yang Ditemukan : RULUDRUR Cost dari solusi : 87 Initial XXXXXXX X0....X X..X..X X....OX X1...LX XZ..X.X XXXXXXX STEP 1 : R XXXXXXX X0....X X..X..X X....OX X1...LX X..ZX.X XXXXXXX STEP 2 : U XXXXXXX X0....X X..X..X X..Z.OX X1...LX X...X.X XXXXXXX STEP 3 : L XXXXXXX X0....X X..X..X

```
XZ...OX
X1...LX
X...X.X
XXXXXXX

STEP 4 : U
XXXXXXX
XZ....X
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 5 : D
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
XZ..X.X
XXXXXXX

STEP 6 : R
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
X..ZX.X
XXXXXXX

STEP 7 : U
XXXXXXX
X0....X
X..X..X
X..Z.OX
X1...LX
X...X.X
XXXXXXX

STEP 8 : R
XXXXXXX
X0....X
X..X..X
X....ZX
X1...LX
```

	X...X.X XXXXXXX Waktu eksekusi: 0.057786 ms Banyak iterasi yang dilakukan: 15 iterasi
Beam-Search	Solusi Yang Ditemukan : RULUDRUR Cost dari solusi : 87 Initial XXXXXXX X0....X X..X..X X....OX X1...LX XZ..X.X XXXXXXX STEP 1 : R XXXXXXX X0....X X..X..X X....OX X1...LX X..ZX.X XXXXXXX STEP 2 : U XXXXXXX X0....X X..X..X X..Z.OX X1...LX X...X.X XXXXXXX STEP 3 : L XXXXXXX X0....X X..X..X XZ...OX X1...LX X...X.X XXXXXXX STEP 4 : U XXXXXXX

```
XZ....X
X..X..X
X....OX
X1...LX
X...X.X
XXXXXXX

STEP 5 : D
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
XZ..X.X
XXXXXXX

STEP 6 : R
XXXXXXX
X0....X
X..X..X
X....OX
X1...LX
X..ZX.X
XXXXXXX

STEP 7 : U
XXXXXXX
X0....X
X..X..X
X..Z.OX
X1...LX
X...X.X
XXXXXXX

STEP 8 : R
XXXXXXX
X0....X
X..X..X
X....ZX
X1...LX
X...X.X
XXXXXXX

Waktu eksekusi: 0.253018 ms
Banyak iterasi yang dilakukan: 15 iterasi
```

Test Case 2

Input:

```
8 8
XXXXXXXX
X***X*LX
X**X***X
X******X
X**X**XX
X*X*ZX*X
XOXXXXXX
XXXXXXXX
999 999 999 999 999 999 999 999
999  8   8   8  999  8   8  999
999  8   8  999  8   8   8  999
999  8   7   8   8   8   8  999
999 999  8   8   8   8   7  999
999  8   8   8   8  999  8  999
999 999 999 999 999 999 999 999
999 999 999 999 999 999 999 999
```

Output:

UCS	Solusi Yang Ditemukan : URDLD Cost dari solusi : 95 Initial XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X.ZX.X XOXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X.LX X..XZ..X X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 2 : R XXXXXXXX

	X...X.LX X..X..ZX X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 3 : D XXXXXXXX X...X.LX X..X...X X.....ZX X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 4 : L XXXXXXXX X...X.LX X..X...X XZ.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 5 : D XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X..X.X XZXXXXXX XXXXXXXX Waktu eksekusi: 0.051196 ms Banyak iterasi yang dilakukan: 7 iterasi
GBFS	Solusi Yang Ditemukan : URDLD Cost dari solusi : 95 Initial XXXXXXXX X...X.LX

```
X..X...X
X......X
X..X..XX
X.X.ZX.X
XOXXXXXX
XXXXXXXX

STEP 1 : U
XXXXXXXX
X...X.LX
X..XZ..X
X......X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 2 : R
XXXXXXXX
X...X.LX
X..X..ZX
X......X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 3 : D
XXXXXXXX
X...X.LX
X..X...X
X.....ZX
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...X.LX
X..X...X
XZ.....X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX
```

	STEP 5 : D XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X..X.X XZXXXXXX XXXXXXXX Waktu eksekusi: 0.05195 ms Banyak iterasi yang dilakukan: 7 iterasi
A*	Solusi Yang Ditemukan : URDLD Cost dari solusi : 95 Initial XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X.ZX.X XOXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X.LX X..XZ..X X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 2 : R XXXXXXXX X...X.LX X..X..ZX X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 3 : D

	XXXXXXXX X...X.LX X..X...X X.....ZX X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 4 : L XXXXXXXX X...X.LX X..X...X XZ.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 5 : D XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X..X.X XZXXXXXX XXXXXXXX Waktu eksekusi: 0.200496 ms Banyak iterasi yang dilakukan: 7 iterasi
BFS	Solusi Yang Ditemukan : URDLD Cost dari solusi : 95 Initial XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X.ZX.X XOXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX

```
X...X.LX
X..XZ..X
X......X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 2 : R
XXXXXXXX
X...X.LX
X..X..ZX
X......X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 3 : D
XXXXXXXX
X...X.LX
X..X...X
X.....ZX
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...X.LX
X..X...X
XZ.....X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
X...X.LX
X..X...X
X......X
X..X..XX
X.X..X.X
XZXXXXXX
XXXXXXXX
```

	Waktu eksekusi: 0.050599 ms Banyak iterasi yang dilakukan: 7 iterasi
Beam-Search	Solusi Yang Ditemukan : URDLD Cost dari solusi : 95 Initial XXXXXXXX X...X.LX X..X...X X.....X X..X..XX X.X.ZX.X XOXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X.LX X..XZ..X X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 2 : R XXXXXXXX X...X.LX X..X..ZX X.....X X..X..XX X.X..X.X XOXXXXXX XXXXXXXX STEP 3 : D XXXXXXXX X...X.LX X..X...X X.....ZX X..X..XX X.X..X.X XOXXXXXX XXXXXXXX

```
STEP 4 : L
XXXXXXXX
X...X.LX
X..X...X
XZ.....X
X..X..XX
X.X..X.X
XOXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
X...X.LX
X..X...X
X......X
X..X..XX
X.X..X.X
XZXXXXXX
XXXXXXXX

Waktu eksekusi: 0.049198 ms
Banyak iterasi yang dilakukan: 7 iterasi
```

Test Case 3

Input:

```
8 8
XXXXXXXX
X***X2*X
X**X0*1X
X*L****X
XX4**3LX
X*O*ZX*X
XXXXXXXX
XXXXXXXX
999 999 999 999 999 999 999 999
999  8   8   8  999  8   8  999
999  8   8  999  8   8   8  999
999  8   7   8   8   8   8  999
999  999 8   8   8   8   7  999
999   8  8   8   8  999  8  999
999 999 999 999 999 999 999 999
999 999 999 999 999 999 999 999
```

Output:

UCS	Solusi Yang Ditemukan : URULDLD Cost dari solusi : 112 Initial XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.O.ZX.X XXXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X2.X X..XZ.1X X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 2 : R XXXXXXXX X...X2.X X..X0.ZX X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 3 : U XXXXXXXX X...X2ZX X..X0.1X X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 4 : L XXXXXXXX X...XZ.X X..X0.1X

	X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 5 : D XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..ZLX X.O..X.X XXXXXXXX XXXXXXXX STEP 6 : L XXXXXXXX X...X2.X X..X0.1X X.L....X XXZ..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 7 : D XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.Z..X.X XXXXXXXX XXXXXXXX Waktu eksekusi: 0.057193 ms Banyak iterasi yang dilakukan: 16 iterasi
GBFS	Solusi Yang Ditemukan : URULDLD Cost dari solusi : 112 Initial XXXXXXXX X...X2.X X..X0.1X X.L....X

```
XX4..3LX
X.O.ZX.X
XXXXXXXX
XXXXXXXX

STEP 1 : U
XXXXXXXX
X...X2.X
X..XZ.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 2 : R
XXXXXXXX
X...X2.X
X..X0.ZX
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 3 : U
XXXXXXXX
X...X2ZX
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...XZ.X
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
```

	X...X2.X X..X0.1X X.L....X XX4..ZLX X.O..X.X XXXXXXXX XXXXXXXX STEP 6 : L XXXXXXXX X...X2.X X..X0.1X X.L....X XXZ..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 7 : D XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.Z..X.X XXXXXXXX XXXXXXXX Waktu eksekusi: 0.076173 ms Banyak iterasi yang dilakukan: 14 iterasi
A*	Solusi Yang Ditemukan : URULDLD Cost dari solusi : 112 Initial XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.O.ZX.X XXXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X2.X

```
X..XZ.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 2 : R
XXXXXXXX
X...X2.X
X..X0.ZX
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 3 : U
XXXXXXXX
X...X2ZX
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...XZ.X
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XX4..ZLX
X.O..X.X
XXXXXXXX
XXXXXXXX
```

	STEP 6 : L XXXXXXXX X...X2.X X..X0.1X X.L....X XXZ..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 7 : D XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.Z..X.X XXXXXXXX XXXXXXXX Waktu eksekusi: 0.132474 ms Banyak iterasi yang dilakukan: 16 iterasi
BFS	Solusi Yang Ditemukan : URULDLD Cost dari solusi : 112 Initial XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.O.ZX.X XXXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X2.X X..XZ.1X X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 2 : R

```
XXXXXXXX
X...X2.X
X..X0.ZX
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 3 : U
XXXXXXXX
X...X2ZX
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...XZ.X
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XX4..ZLX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 6 : L
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XXZ..3LX
X.O..X.X
XXXXXXXX
```

	XXXXXXXX STEP 7 : D XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.Z..X.X XXXXXXXX XXXXXXXX Waktu eksekusi: 0.050721 ms Banyak iterasi yang dilakukan: 16 iterasi
Beam-Search	Solusi Yang Ditemukan : URULDLD Cost dari solusi : 112 Initial XXXXXXXX X...X2.X X..X0.1X X.L....X XX4..3LX X.O.ZX.X XXXXXXXX XXXXXXXX STEP 1 : U XXXXXXXX X...X2.X X..XZ.1X X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX STEP 2 : R XXXXXXXX X...X2.X X..X0.ZX X.L....X XX4..3LX X.O..X.X XXXXXXXX XXXXXXXX

```
STEP 3 : U
XXXXXXXX
X...X2ZX
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 4 : L
XXXXXXXX
X...XZ.X
X..X0.1X
X.L....X
XX4..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 5 : D
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XX4..ZLX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 6 : L
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XXZ..3LX
X.O..X.X
XXXXXXXX
XXXXXXXX

STEP 7 : D
XXXXXXXX
X...X2.X
X..X0.1X
X.L....X
XX4..3LX
```

	X.Z..X.X XXXXXXXX XXXXXXXX Waktu eksekusi: 0.057582 ms Banyak iterasi yang dilakukan: 8 iterasi

Test Case 4

Input:

```
10 10
*******X**
*X***X****
***X******
********X*
**X*******
*******X**
****X*****
*Z******X*
*******OX*
******XX**
 1   2   3   4   5   6   7  999  8   9
 1  999  2   3   4  999  5   6   7   8
 1   2   3   4  999  5   6   7   8   9
 1   2   3   4   5   6   7   8  999  0
 1   2  999  3   4   5   6   7   8   9
 1   2   3   4   5   6   7  999  8   9
 1   2   3   4  999  5   6   7   8   9
 1   2   3   4  990  6   7   8  999  9
 1   2   3   4   5   6   7   8  999  0
 1   2   3   4   5   6  999 999  9   0
```

Output:

UCS	Solusi Yang Ditemukan : URDRULDR Cost dari solusi : 0 Initial .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .Z.....X.

```
.......OX.
......XX..

STEP 1 : U
.......X..
.X...X....
.Z.X......
........X.
..X.......
.......X..
....X.....
........X.
.......OX.
......XX..

STEP 2 : R
.......X..
.X...X....
..ZX......
........X.
..X.......
.......X..
....X.....
........X.
.......OX.
......XX..

STEP 3 : D
.......X..
.X...X....
...X......
..Z.....X.
..X.......
.......X..
....X.....
........X.
.......OX.
......XX..

STEP 4 : R
.......X..
.X...X....
...X......
.......ZX.
..X.......
.......X..
....X.....
```

```
........X.
.......OX.
......XX..

STEP 5 : U
.......X..
.X...X.Z..
...X......
........X.
..X.......
.......X..
....X.....
........X.
.......OX.
......XX..

STEP 6 : L
.......X..
.X...XZ...
...X......
........X.
..X.......
.......X..
....X.....
........X.
.......OX.
......XX..

STEP 7 : D
.......X..
.X...X....
...X......
........X.
..X.......
.......X..
....X.....
........X.
......ZOX.
......XX..

STEP 8 : R
.......X..
.X...X....
...X......
........X.
..X.......
.......X..
```

	....X..... .....X. .....ZX. .....XX.. Waktu eksekusi: 0.05111 ms Banyak iterasi yang dilakukan: 9 iterasi
GBFS	Solusi Yang Ditemukan : RD Cost dari solusi : 1 Initial .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .Z.....X. .....OX. .....XX.. STEP 1 : R .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....ZX. .....OX. .....XX.. STEP 2 : D .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....X. .....ZX. .....XX..

	Waktu eksekusi: 0.053499 ms Banyak iterasi yang dilakukan: 3 iterasi
A*	Solusi Yang Ditemukan : RD Cost dari solusi : 1 Initial .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .Z.....X. .....OX. .....XX.. STEP 1 : R .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....ZX. .....OX. .....XX.. STEP 2 : D .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....X. .....ZX. .....XX.. Waktu eksekusi: 0.038876 ms Banyak iterasi yang dilakukan: 3 iterasi
BFS	Solusi Yang Ditemukan : RD Cost dari solusi : 1

	Initial .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .Z.....X. .....OX. .....XX.. STEP 1 : R .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....ZX. .....OX. .....XX.. STEP 2 : D .....X.. .X...X.... ...X..... .....X. ..X..... .....X.. ....X..... .....X. .....ZX. .....XX.. Waktu eksekusi: 0.069939 ms Banyak iterasi yang dilakukan: 5 iterasi
Beam-Search	Solusi Yang Ditemukan : RD Cost dari solusi : 1 Initial .....X.. .X...X.... ...X.....

```
........X.
..X.......
.......X..
....X.....
.Z......X.
.......OX.
......XX..

STEP 1 : R
.......X..
.X...X....
...X......
........X.
..X.......
.......X..
....X.....
.......ZX.
.......OX.
......XX..

STEP 2 : D
.......X..
.X...X....
...X......
........X.
..X.......
.......X..
....X.....
........X.
.......ZX.
......XX..

Waktu eksekusi: 0.048497 ms
Banyak iterasi yang dilakukan: 3 iterasi
```

Test Case 5

Input:

```
10 10
**X*X**X**
*XO**X****
***X******
********X*
*1X*******
X***0**X**
****X*****
```

```
*Z******X*
**********
*******X**
 1   2  999  4   5   6   7  999  8   9
 1  999  2  999  4  999  5   6   7   8
 1   2   3   4  999  5   6   7   8   9
 1   2   3   4   5   6   7   8  999  0
 1   2  999  3   4   5   6   7   8   9
 1  999  3   4   5   6   7  999  8   9
 1   2   3   4  999  5   6   7   8   9
 1   2   3   4   5   6   7   8  999  9
 1   2   3   4   5   6   7   8   9   0
 1   2   3   4   5   6   7  999  9   0
```

Output:

UCS	Solusi Yang Ditemukan : RULULDLURU Cost dari solusi : 105 Initial ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .Z.....X. ....X.. STEP 1 : R ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....ZX. ....X.. STEP 2 : U ..X.X..X.. .XO..X.... ...X.....

```
........X.
.1X.......
X...0..X..
....X..Z..
........X.
..........
.......X..

STEP 3 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...0..X..
....XZ....
........X.
..........
.......X..

STEP 4 : U
..X.X..X..
.XO..X....
...X.Z....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 5 : L
..X.X..X..
.XO..X....
...XZ.....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 6 : D
..X.X..X..
.XO..X....
```

```
...X......
........X.
.1X.......
X...Z..X..
....X.....
........X.
..........
.......X..

STEP 7 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
XZ..0..X..
....X.....
........X.
..........
.......X..

STEP 8 : U
..X.X..X..
.XO..X....
.Z.X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 9 : R
..X.X..X..
.XO..X....
..ZX......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 10 : U
..X.X..X..
```

	.XZ..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....X. ....X.. Waktu eksekusi: 0.083982 ms Banyak iterasi yang dilakukan: 18 iterasi
GBFS	Solusi Yang Ditemukan : RULULDLURU Cost dari solusi : 105 Initial ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .Z.....X. ....X.. STEP 1 : R ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....ZX. ....X.. STEP 2 : U ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X..

```
....X..Z..
........X.
..........
.......X..

STEP 3 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...0..X..
....XZ....
........X.
..........
.......X..

STEP 4 : U
..X.X..X..
.XO..X....
...X.Z....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 5 : L
..X.X..X..
.XO..X....
...XZ.....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 6 : D
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
```

```
X...Z..X..
....X.....
........X.
..........
.......X..

STEP 7 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
XZ..0..X..
....X.....
........X.
..........
.......X..

STEP 8 : U
..X.X..X..
.XO..X....
.Z.X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 9 : R
..X.X..X..
.XO..X....
..ZX......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 10 : U
..X.X..X..
.XZ..X....
...X......
........X.
```

	.1X..... X...0..X.. ....X..... .....X. ....X.. Waktu eksekusi: 0.668874 ms Banyak iterasi yang dilakukan: 17 iterasi
A*	Solusi Yang Ditemukan : RULULDLURU Cost dari solusi : 105 Initial ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .Z.....X. ....X.. STEP 1 : R ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....ZX. ....X.. STEP 2 : U ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..Z.. .....X. .....

```
.......X..

STEP 3 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...0..X..
....XZ....
........X.
..........
.......X..

STEP 4 : U
..X.X..X..
.XO..X....
...X.Z....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 5 : L
..X.X..X..
.XO..X....
...XZ.....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 6 : D
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...Z..X..
....X.....
........X.
```

```
..........
.......X..

STEP 7 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
XZ..0..X..
....X.....
........X.
..........
.......X..

STEP 8 : U
..X.X..X..
.XO..X....
.Z.X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 9 : R
..X.X..X..
.XO..X....
..ZX......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 10 : U
..X.X..X..
.XZ..X....
...X......
........X.
.1X.......
X...0..X..
....X.....
```

	.....X. ....X.. Waktu eksekusi: 0.209895 ms Banyak iterasi yang dilakukan: 17 iterasi
BFS	Solusi Yang Ditemukan : RULULDLURU Cost dari solusi : 105 Initial ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .Z.....X. ....X.. STEP 1 : R ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....ZX. ....X.. STEP 2 : U ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..Z.. .....X. ....X.. STEP 3 : L

```
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...0..X..
....XZ....
........X.
..........
.......X..

STEP 4 : U
..X.X..X..
.XO..X....
...X.Z....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 5 : L
..X.X..X..
.XO..X....
...XZ.....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 6 : D
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...Z..X..
....X.....
........X.
..........
.......X..
```

```
STEP 7 : L
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
XZ..0..X..
....X.....
........X.
..........
.......X..

STEP 8 : U
..X.X..X..
.XO..X....
.Z.X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 9 : R
..X.X..X..
.XO..X....
..ZX......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 10 : U
..X.X..X..
.XZ..X....
...X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..
```

	Waktu eksekusi: 0.168116 ms Banyak iterasi yang dilakukan: 21 iterasi
Beam-Search	Solusi Yang Ditemukan : RULULDLURU Cost dari solusi : 105 Initial ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .Z.....X. ....X.. STEP 1 : R ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..... .....ZX. ....X.. STEP 2 : U ..X.X..X.. .XO..X.... ...X..... .....X. .1X..... X...0..X.. ....X..Z.. .....X. ....X.. STEP 3 : L ..X.X..X.. .XO..X.... ...X.....

```
........X.
.1X.......
X...0..X..
....XZ....
........X.
..........
.......X..

STEP 4 : U
..X.X..X..
.XO..X....
...X.Z....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 5 : L
..X.X..X..
.XO..X....
...XZ.....
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 6 : D
..X.X..X..
.XO..X....
...X......
........X.
.1X.......
X...Z..X..
....X.....
........X.
..........
.......X..

STEP 7 : L
..X.X..X..
.XO..X....
```

```
...X......
........X.
.1X.......
XZ..0..X..
....X.....
........X.
..........
.......X..

STEP 8 : U
..X.X..X..
.XO..X....
.Z.X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 9 : R
..X.X..X..
.XO..X....
..ZX......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

STEP 10 : U
..X.X..X..
.XZ..X....
...X......
........X.
.1X.......
X...0..X..
....X.....
........X.
..........
.......X..

Waktu eksekusi: 0.107272 ms
Banyak iterasi yang dilakukan: 11 iterasi
```

ANALISIS PENGUJIAN

Pada percobaan pertama, terlihat bahwa algoritma *UCS*, *A**, *BFS*, dan *Beam Search* menghasilkan solusi yang sama yaitu *RULUDRUR* dengan biaya 87. Namun, algoritma *GBFS* menghasilkan solusi *RURULDRUR* dengan biaya lebih tinggi yaitu 104. Hal ini menunjukkan bahwa heuristik yang digunakan kurang tepat memprediksi biaya estimasi. Hal ini sesuai dengan *trade-off* algoritma *GBFS* yang tidak selalu menghasilkan solusi optimal.

Pengujian kedua dan ketiga cukup sama. Kelima algoritma menghasilkan solusi yang sama persis untuk masing-masing pengujian. Namun, terdapat perbedaan pada banyak iterasi tiap algoritma pada pengujian tiga. Algoritma *UCS*, *A**, dan *BFS* menghasilkan banyak iterasi yang sama yaitu 16, *GBFS* menghasilkan iterasi sebanyak 14, sedangkan *Beam Search* sebanyak 8 saja. Hal ini menunjukkan bahwa *Beam Search* lebih efisien proses pencarian solusinya dari yang lainnya dan disusul oleh *GBFS*. Kedua algoritma ini tidak selalu menghasilkan solusi optimal sebagai *trade off* dari efisiensi pencariannya.

Pengujian keempat cukup menarik di bagian solusi dari *UCS* dan *A**. Terlihat bahwa jalur yang diambil *A** lebih mahal biayanya daripada jalur yang diambil *UCS*. Hal ini sesuai dengan ketidak-*admissible*-an *A** ketika terdapat petak dengan biaya nol yang dijelaskan pada bab penjelasan algoritma bagian *A**.

Terakhir, pada pengujian kelima mirip seperti pengujian ketiga. Kelima algoritma menghasilkan solusi yang sama, namun iterasi yang dilakukan berbeda-beda. Terlihat bahwa *A** memiliki iterasi yang lebih sedikit, yaitu 17, dari *UCS*, yaitu 18. Hal ini karena *A** mengutilisasi sifat *GBFS* sehingga lebih efisien dalam menentukan simpul terbaik. *GBFS* juga demikian, memiliki iterasi yang sama dengan *A**. Algoritma *BFS* memiliki iterasi terbanyak, yaitu 21, karena *BFS* tidak memiliki algoritma khusus untuk menentukan simpul mana yang terbaik untuk ditelusuri. Algoritma *Beam Search* memiliki iterasi terkecil, yaitu 11 iterasi saja, jauh di bawah algoritma lainnya. Hal ini menunjukkan bahwa *Beam Search* adalah algoritma paling efisien dalam pencarian solusi pada pengujian ini.

KESIMPULAN

Berbagai algoritma penentuan rute dapat diimplementasikan dalam penentuan solusi pada suatu gim, seperti pada contoh ini yaitu gim Ice Sliding Puzzle. Algoritma-algoritma tersebut berhasil diimplementasikan dalam bentuk kode program dan menghasilkan keluaran yang cukup sesuai dengan ekspektasi. Tidak ada algoritma yang terbaik secara umum, melainkan setiap algoritma memiliki kelebihan dan kekurangannya masing-masing. Penentuan algoritma terbaik pada situasi tertentu dapat dilakukan dengan melihat kekomplitan, keoptimalan solusi, dan keefisienan pencarian yang diharapkan sehingga didapatkan algoritma terbaik pada situasi tersebut.

SARAN

Kami sangat berterima kasih kepada para asisten yang telah menyiapkan pelaksanaan tugas kecil ini. Namun, beberapa hal masih terasa kurang memuaskan dari kami sebagai mahasiswa yang mengerjakannya. Salah satunya adalah ketidakjelasan dokumen spesifikasi terhadap apa yang ingin dikeluarkan oleh program yang ditambah dengan ketidakkonsistenan jawaban pertanyaan pada *QnA* terhadap dokumen spesifikasi. Semoga, di tugas berikutnya, ketidakjelasan spesifikasi ini bisa berkurang dan jawaban *QnA* bisa lebih sesuai dengan spesifikasi yang ada.

PERNYATAAN INTEGRITAS

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Faiq Azzam Nafidz

13524003

Billie Bhaskara Wibawa

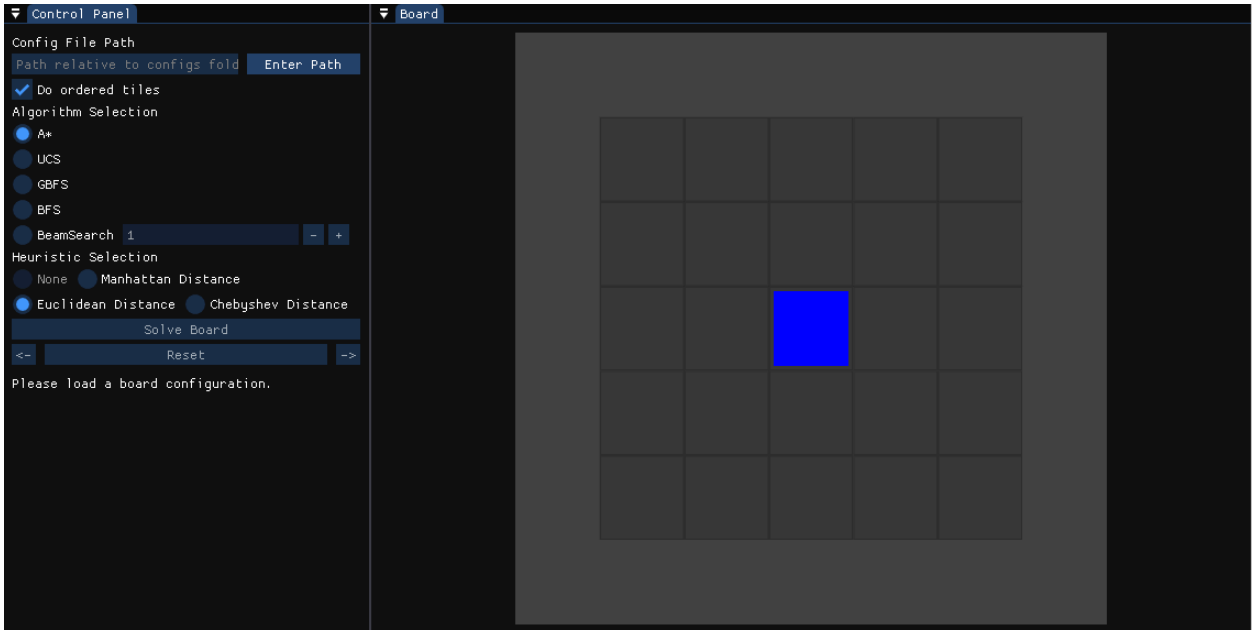
13524024

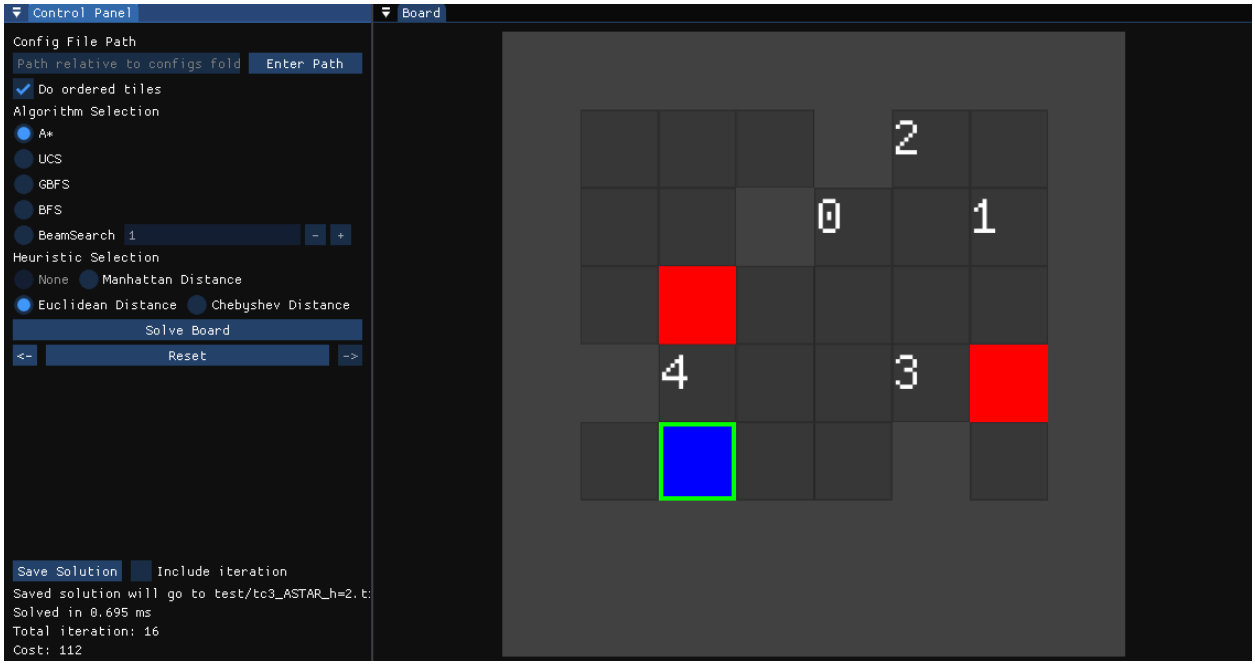
No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma *pathfinding* alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

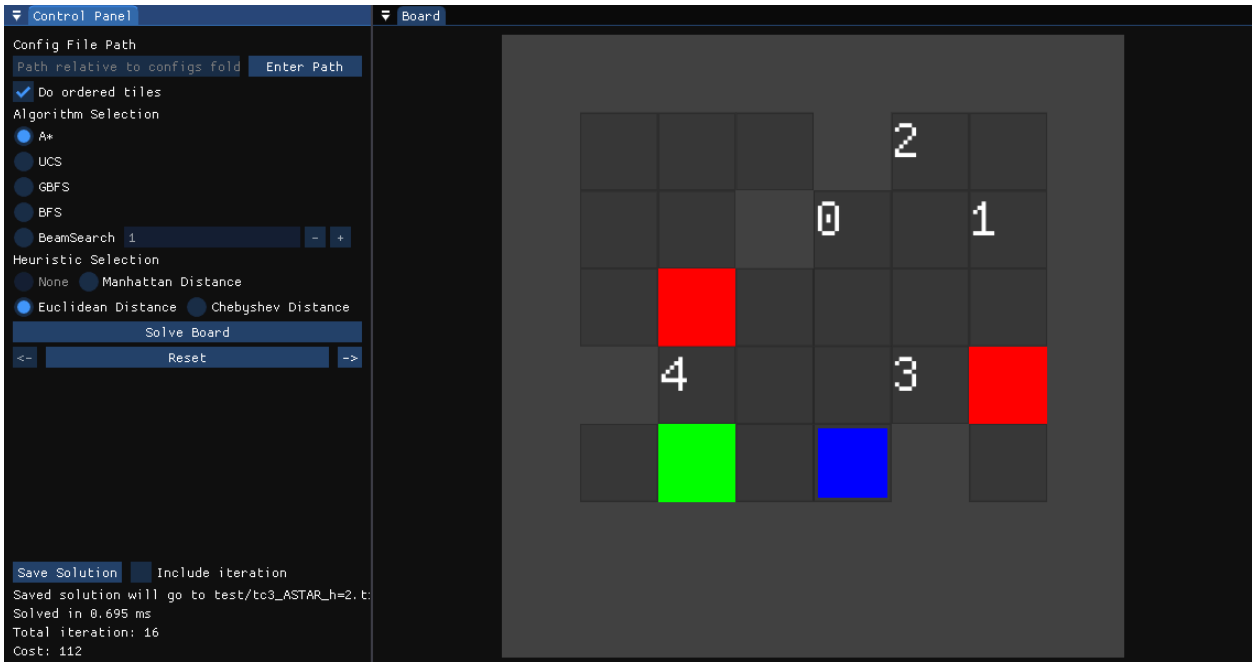
Tabel *Checklist* fitur

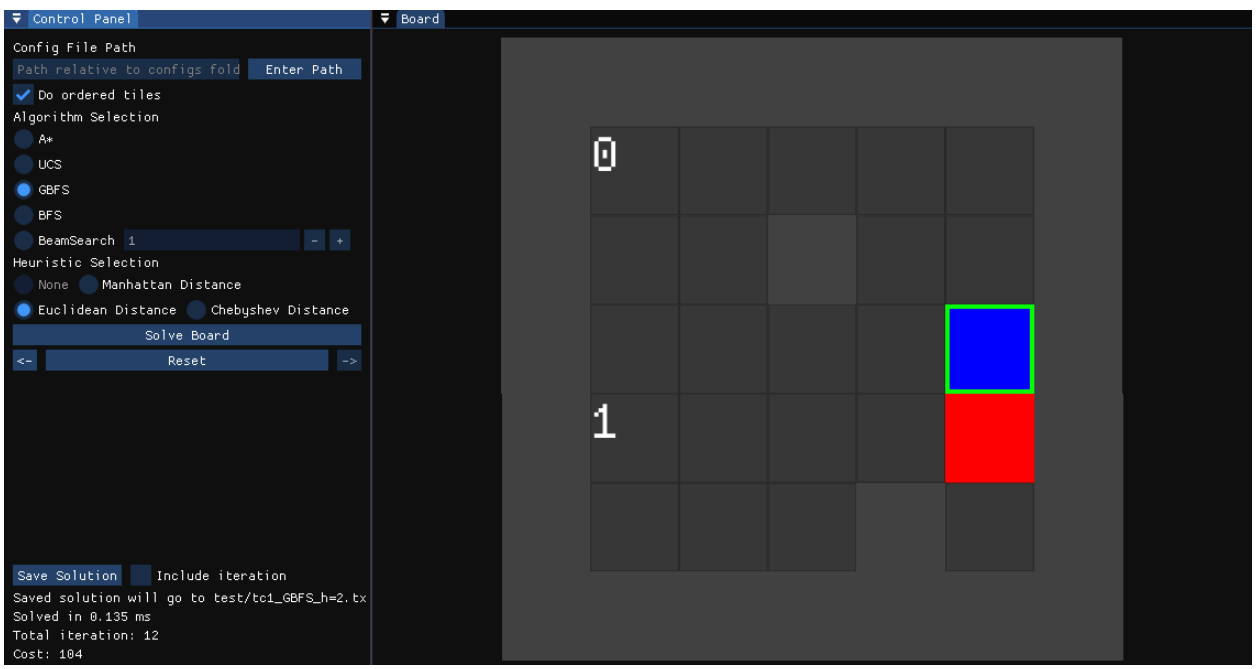
https://github.com/billie-bytes/Tucil3_13524003_13524024

Repository kode sumber aplikasi.


Control Panel
Board
Config File Path
Path relative to configs fold
Enter Path
Do ordered tiles
Algorithm Selection
A*
UCS
GBFS
BFS
BeamSearch
1
-
+
Heuristic Selection
None
Manhattan Distance
Euclidean Distance
Chebyshev Distance
Solve Board
<-
Reset
->
Please load a board configuration.


Control Panel
Board
Config File Path
Path relative to configs fold
Enter Path
Do ordered tiles
Algorithm Selection
A*
UCS
GBFS
BFS
BeamSearch
1
-
+
Heuristic Selection
None
Manhattan Distance
Euclidean Distance
Chebyshev Distance
Solve Board
<-
Reset
->
2
0
1
4
3
Save Solution
Include iteration
Saved solution will go to test/tc3_ASTAR_h=2.t
Solved in 0.695 ms
Total iteration: 16
Cost: 112


Control Panel
Board
Config File Path
Path relative to configs fold
Enter Path
Do ordered tiles
Algorithm Selection
A*
UCS
GBFS
BFS
BeamSearch
1
Heuristic Selection
None
Manhattan Distance
Euclidean Distance
Chebyshev Distance
Solve Board
<-
Reset
->
Save Solution
Include iteration
Saved solution will go to test/tc3_ASTAR_h=2.t
Solved in 0.695 ms
Total iteration: 16
Cost: 112
2
0
1
4
3


Control Panel
Board
Config File Path
Path relative to configs fold
Enter Path
Do ordered tiles
Algorithm Selection
A*
UCS
GBFS
BFS
BeamSearch
1
Heuristic Selection
None
Manhattan Distance
Euclidean Distance
Chebyshev Distance
Solve Board
<-
Reset
->
Save Solution
Include iteration
Saved solution will go to test/tc1_GBFS_h=2.tx
Solved in 0.135 ms
Total iteration: 12
Cost: 104
0
1

Control Panel
Board
Config File Path
Path relative to configs fold
Enter Path
Do ordered tiles
Algorithm Selection
A*
UCS
GBFS
BFS
BeamSearch
1
-
+
Heuristic Selection
None
Manhattan Distance
Euclidean Distance
Chebyshev Distance
Solve Board
<-
Reset
->
Failed to find a solution!
Execution time: 0.068 ms
0
1